

Deep Generative Models

Chapter 6: Generation by Diffusion Process

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Summer 2026

Diffusion Probabilistic Model

Diffusion Probabilistic Models (DPMs)

DPMs learn the reverse diffusion process by considering a **Gaussian** conditional distribution, i.e., they assume

$$P_{\mathbf{w}}(x_{t-1}|x_t) \equiv \mathcal{N}(\mu_{\mathbf{w}}(x_t, t), \Sigma_{\mathbf{w}}(x_t, t))$$

for some computational models $\mu_{\mathbf{w}}$ and $\Sigma_{\mathbf{w}}$

- + Do DPMs really use $\Sigma_{\mathbf{w}}$?! You said we **only** learn the mean!
- **Not really in practice!**

Most practical DPMs **only** learn the mean value

$$P_{\mathbf{w}}(x_{t-1}|x_t) \equiv \mathcal{N}(\mu_{\mathbf{w}}(x_t, t), \sigma_t^2)$$

for some predefined σ_t

Visualizing a DPM: Additive Gaussian Noise

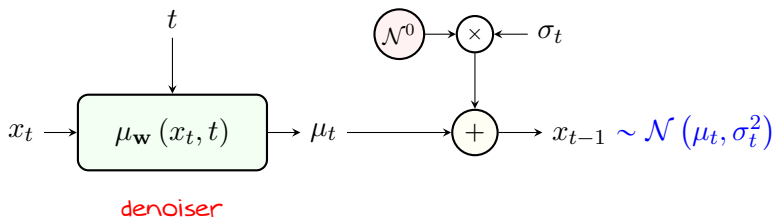
The reverse process is *dual of the forward diffusion*: we can see

$$P_{\mathbf{w}}(x_{t-1}|x_t) \equiv \mathcal{N}(\mu_{\mathbf{w}}(x_t, t), \sigma_t^2)$$

as a standard Gaussian process with **additive noise**

$$x_{t-1} = \mu_{\mathbf{w}}(x_t, t) + \sigma_t \tilde{\epsilon}_t$$

This is very easy to implement by a **denoising** network!



Generation by DPMs

- + *How do DPMs generate a new data sample?*
- Just move on the reverse process back in time

We can sample **Gaussian noise** and move reversely in time

$$x_0 \xleftarrow{P_{\mathbf{w},1}} x_1 \xleftarrow{P_{\mathbf{w},2}} \dots \xleftarrow{P_{\mathbf{w},t}} x_{t-1} \xleftarrow{P_{\mathbf{w},t}} x_t \xleftarrow{P_{\mathbf{w},T}} \dots \xleftarrow{P_{\mathbf{w},T}} x_T$$

Sample_DPM($\mu_{\mathbf{w}}$:Denoiser)

- 1: Sample **latent** $x_T \sim \mathcal{N}^0$
- 2: **for** $t = T, \dots, 1$ **do**
- 3: Pass x_t and t forward to find $\mu_t = \mu_{\mathbf{w}}(x_t, t)$
- 4: Sample $\varepsilon \sim \mathcal{N}^0$
- 5: Denoise as $x_{t-1} = \mu_t + \sigma_t \varepsilon$
- 6: **end for**
- 7: **return** **Sample** x_0

DPM Training: Naive Training

- + How do we train DPMs?!
- We should only train the **denoiser**

Following **ELBO maximization**: we compute *empirical risk* as

$$\begin{aligned}\hat{R}(\mathbf{w}) &= \hat{\mathbb{E}}_{x_0 \sim P_{\text{data}}} \{R(\mathbf{w}|x_0)\} \\ &= \hat{\mathbb{E}}_{x_0 \sim P_{\text{data}}} \left\{ \sum_{t=1}^T \frac{1}{\sigma_t^2} \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2 \right\}\end{aligned}$$

And, just to recall: η_t is the posterior mean computed as

$$\eta_t = \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

DPM Training: Naive Training

We could define $\omega_t = 1/\sigma_t^2$ and write the risk function compactly as

$$\hat{R}(\mathbf{w}) = \hat{\mathbb{E}}_{x_0 \sim P_{\text{data}}} \left\{ \sum_{t=1}^T \omega_t \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2 \right\}$$

We can then write a training loop to minimize this *empirical risk*

- + What is the order of T ?!
 - T is rather large, e.g., $T = 1000$!
- 🕒 This sounds like a *huge* forward and backward pass at *every sample*!!
 - Totally agree! But, there is a way *around it*

DPM Training: *Time Sampling*

There is no harm to scale $\hat{R}(\mathbf{w})$ by a *constant*!

$$\begin{aligned}\hat{R}(\mathbf{w}) &= \frac{1}{T} \hat{\mathbb{E}}_{x_0 \sim P_{\text{data}}} \left\{ \sum_{t=1}^T \omega_t \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2 \right\} \\ &= \hat{\mathbb{E}}_{x_0 \sim P_{\text{data}}} \left\{ \frac{1}{T} \sum_{t=1}^T \omega_t \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2 \right\}\end{aligned}$$

Now, assuming that $t \sim \mathcal{U}_T$ is *uniformly* distributed on $\{1, \dots, T\}$, we could say

$$\hat{R}(\mathbf{w}) = \hat{\mathbb{E}}_{x_0 \sim P_{\text{data}}} \left\{ \mathbb{E}_{t \sim \mathcal{U}_T} \left\{ \omega_t \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2 \right\} \right\}$$

DPM Loss Computation by *Time Sampling*

As always, we can estimate *time expectation* by *sampling*: let the loss function for a single *data* and *time* sample be

$$R(\mathbf{w}|x_0, t) = \omega_t \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2$$

We could say that

$$\mathbb{E}_{t \sim \mathcal{U}_T} \{\omega_t \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2\} \approx \hat{\mathbb{E}}_{t \sim \mathcal{U}_T} \{\omega_t \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2\}$$

where for computing *right-hand-side*, we *sample* *t* *uniformly* from $\{1, \dots, T\}$

The *empirical risk* is then estimated as

$$\hat{R}(\mathbf{w}) = \hat{\mathbb{E}}_{x_0, t} \{\omega_t \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2\}$$

DPM Loss Computation: Pseudo Code

It might be easier to think of it algorithmically

```
Grad_Compute_DPM( $\mu_w$ :Denoiser,  $\mathbb{B}$ :data_batch)
```

- 1: **for** each *sample* j in $\mathbb{B}$ **do**
- 2: Sample t^j *uniformly from* $\{1, \dots, T\}$
- 3: Sample $\varepsilon \sim \mathcal{N}^0$ and compute $x_{t^j}^j = \sqrt{\bar{\alpha}_{t^j}} x_0^j + \sqrt{1 - \bar{\alpha}_{t^j}} \varepsilon$
- 4: Compute $\eta_{t^j}^j$ from x_0^j and $x_{t^j}^j$
- 5: Pass x_{t^j} and t^j forward to find $\mu_{t^j} = \mu_w(x_{t^j}, t^j)$
- 6: Compute sample loss as $R_{t^j}^j = \omega_{t^j} \|\mu_{t^j} - \eta_{t^j}^j\|^2$
- 7: Backpropagate to compute $\nabla R_{t^j}^j$
- 8: **end for**
- 9: Estimate empirical risk as $\hat{R} = \text{mean}(R_{t^1}^1, \dots, R_{t^n}^n)$
- 10: Estimate batch gradient as $\nabla \hat{R} = \text{opt_avg}\{R_{t^1}^1, \dots, R_{t^n}^n\}$
- 11: **return** $\hat{R}$ and $\nabla \hat{R}$

Training DPMs: *Efficient Approach*

We can use this trick to build an efficient training loop

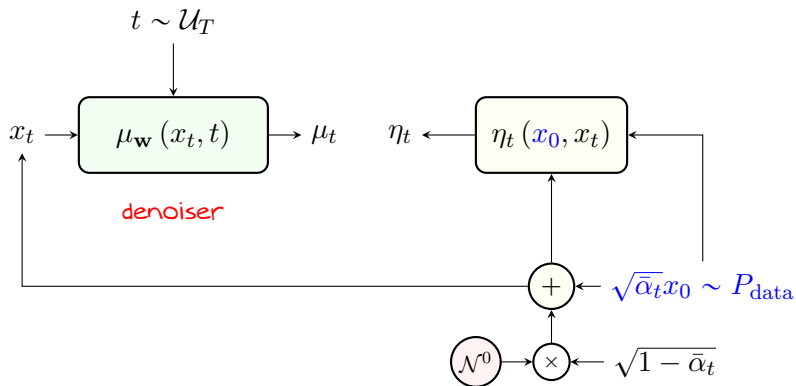
```
Training_DPM( $\mu_{\mathbf{w}}$ :Denoiser,  $\mathbb{D}$ :dataset)
```

```
1: Initiate  $\mu_{\mathbf{w}}$  with some weights  
2: for multiple epochs do  
3:   Sample a batch  $\mathbb{B}$  from  $\mathbb{D}$   
4:   Compute batch gradient as  $\hat{R}, \nabla \hat{R} \leftarrow \text{Grad\_Compute\_DPM}(\mu_{\mathbf{w}}, \mathbb{B})$   
5:   Update the weights as  $\mathbf{w} \leftarrow \mathbf{w} - \nabla \hat{R}$   
6: end for  
7: return  $\mu_{\mathbf{w}}$ 
```

and we could use the trained model to generate a new data sample

$$x_{\text{new}} \leftarrow \text{Sample_DPM}(\mu_{\mathbf{w}})$$

DPM Training Loop: Visualization



Posterior Mean: *Denoising Interpretation*

Let's take a look at *posterior mean* that we use for training

$$\eta_t = \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t$$

We also recall that

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t$$

So, we can expand the term as

$$\begin{aligned}\eta_t &= \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t) \\ &= \underbrace{\frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t) + \sqrt{\bar{\alpha}_t}\alpha_t(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}}_{a_t}x_0 + \underbrace{\frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{\sqrt{1 - \bar{\alpha}_t}}}_{b_t}\epsilon_t\end{aligned}$$

Posterior Mean: *Denoising Interpretation*

We can use the fact that

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i \rightsquigarrow \bar{\alpha}_t = \bar{\alpha}_{t-1} \alpha_t$$

to *simplify* the coefficients: for α_t we have

$$\begin{aligned} \alpha_t &= \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_t} [1 - \alpha_t + \alpha_t (1 - \bar{\alpha}_{t-1})] \\ &= \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_t} (1 - \bar{\alpha}_t) \\ &= \sqrt{\bar{\alpha}_{t-1}} = \frac{\sqrt{\bar{\alpha}_t}}{\sqrt{\alpha_t}} \end{aligned}$$

Posterior Mean: *Denoising Interpretation*

Similarly for b_t we can write

$$\begin{aligned}
 b_t &= \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{\sqrt{1 - \bar{\alpha}_t}} \\
 &= \frac{1}{\sqrt{\alpha_t}} \frac{\alpha_t (1 - \bar{\alpha}_{t-1})}{\sqrt{1 - \bar{\alpha}_t}} \\
 &= \frac{1}{\sqrt{\alpha_t}} \frac{\alpha_t - \bar{\alpha}_t + 1 - 1}{\sqrt{1 - \bar{\alpha}_t}} \\
 &= \frac{1}{\sqrt{\alpha_t}} \frac{1 - \bar{\alpha}_t - (1 - \alpha_t)}{\sqrt{1 - \bar{\alpha}_t}} \\
 &= \frac{1}{\sqrt{\alpha_t}} \left(\sqrt{1 - \bar{\alpha}_t} - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \right)
 \end{aligned}$$

Posterior Mean: *Denoising Interpretation*

Putting them together: we can write η_t as

$$\begin{aligned}
 \eta_t &= a_t \mathbf{x}_0 + b_t \boldsymbol{\varepsilon}_t \\
 &= \frac{\sqrt{\bar{\alpha}_t}}{\sqrt{\alpha_t}} \mathbf{x}_0 + \frac{1}{\sqrt{\alpha_t}} \left(\sqrt{1 - \bar{\alpha}_t} - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \right) \boldsymbol{\varepsilon}_t \\
 &= \frac{1}{\sqrt{\alpha_t}} \left(\underbrace{\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}_t}_{\mathbf{x}_t} - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\varepsilon}_t \right) \\
 &= \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\varepsilon}_t \right) \equiv \text{denoiser of } \mathbf{x}_t
 \end{aligned}$$

η_t is indeed a *denoising* of $\mathbf{x}_t \rightsquigarrow$ **Bingo!** This is why we called $\mu_{\mathbf{w}}$ *denoiser*

Connection to Direct Denoiser

We could also *denoise* simply by *reversing the forward diffusion*

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\tilde{\epsilon}_t$$

! Attention

$\tilde{\epsilon}_t$ is noise sample *added in step t* of forward diffusion

↳ It is different from ϵ_t which builds the *direct link*

↳ They are though *correlated!*

We can directly *denoise* by *reversing* this equation as

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} (x_t - \sqrt{1 - \alpha_t}\tilde{\epsilon}_t)$$

Connection to *Direct Denoiser*

Indeed, we could think of η_t as same *reverse diffusion* when

we use an estimator of $\tilde{\epsilon}_t$ computed from ϵ_t as

$$\hat{\epsilon}_t = \sqrt{\frac{1 - \alpha_t}{1 - \bar{\alpha}_t}} \epsilon_t$$

Using this estimator, we can estimate the *reverse diffusion* as

$$\begin{aligned} \hat{x}_{t-1} &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \sqrt{1 - \alpha_t} \hat{\epsilon}_t \right) \\ &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) = \eta_t \end{aligned}$$

Key Observation: *Sample Loss*

Now, let us look at the training again: we compute *sample loss* as

$$R(\mathbf{w}|x_0, t) = \omega_t \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2$$

and we know that

$$\eta_t = \hat{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_t \right)$$

Interesting Observation

When we train DPM, we have ε_t , since we use it to sample x_t from x_0

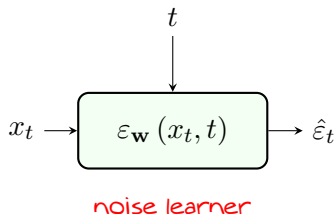
This observation leads to introduction of *denoising DPMs*

DDPM: Specific Choice of DPM

Denoising DPM (DDPM)

In DDPMs, we set the model to estimate *direct noise* at time t and let

$$\mu_{\mathbf{w}}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\mathbf{w}}(x_t, t) \right)$$



Sample Loss of DDPM

Using DDPM: the *sample loss* becomes

$$\begin{aligned}
 R(\mathbf{w}|x_0, t) &= \omega_t \|\mu_{\mathbf{w}}(x_t, t) - \eta_t\|^2 \\
 &= \omega_t \underbrace{\frac{(1 - \alpha_t)^2}{\alpha_t (1 - \bar{\alpha}_t)}}_{\bar{\omega}_t} \|\varepsilon_{\mathbf{w}}(x_t, t) - \varepsilon_t\|^2 \\
 &= \bar{\omega}_t \|\varepsilon_{\mathbf{w}}(x_t, t) - \varepsilon_t\|^2
 \end{aligned}$$

which we can *easily compute!*

Intuitive Interpretation

We train a *model* to estimate *direct noise sample* from *noisy data*

- ↳ We use the model to *estimate direct noise* in each *time step*
- ↳ We *denoise* to *reverse the diffusion* time step

DDPM: Risk Computation

We can build a similar loop to compute empirical risk

```
Grad_Compute_DDPM( $\varepsilon_{\mathbf{w}}$ :Denoiser,  $\mathbb{B}$ :data_batch)
```

```

1: for each sample  $x_0^j$  in  $\mathbb{B}$  do
2:   Sample  $t^j$  uniformly from  $\{1, \dots, T\}$ 
3:   Sample  $\varepsilon \sim \mathcal{N}^0$  and compute  $x_{t^j}^j = \sqrt{\bar{\alpha}_{t^j}} x_0^j + \sqrt{1 - \bar{\alpha}_{t^j}} \varepsilon$ 
4:   Pass  $x_{t^j}$  and  $t^j$  forward to find  $\hat{\varepsilon}_{t^j} = \varepsilon_{\mathbf{w}}(x_{t^j}, t^j)$ 
5:   Compute sample loss as  $R_{t^j}^j = \bar{\omega}_{t^j} \|\hat{\varepsilon}_{t^j} - \varepsilon\|^2$ 
6:   Backpropagate to compute  $\nabla R_{t^j}^j$ 
7: end for
8: Estimate empirical risk as  $\hat{R} = \text{mean}(R_{t^1}^1, \dots, R_{t^n}^n)$ 
9: Estimate gradient as  $\nabla \hat{R} = \text{0pt\_avg}\{\nabla R_{t^1}^1, \dots, \nabla R_{t^n}^n\}$ 
10: return  $\hat{R}$  and  $\nabla \hat{R}$ 
```

DDPM: Training Loop

Training_DDPM($\varepsilon_{\mathbf{w}}$:Denoiser, $\mathbb{D}$:dataset)

- 1: *Initiate $\varepsilon_{\mathbf{w}}$ with some weights*
- 2: **for** *multiple epochs* **do**
- 3: *Sample a batch $\mathbb{B}$ from $\mathbb{D}$*
- 4: *Compute batch gradient as $\hat{R}, \nabla \hat{R} \leftarrow \text{Grad_Compute_DDPM}(\varepsilon_{\mathbf{w}}, \mathbb{B})$*
- 5: *Update the weights as $\mathbf{w} \leftarrow \mathbf{w} - \nabla \hat{R}$*
- 6: **end for**
- 7: **return** $\varepsilon_{\mathbf{w}}$

DDPM: Generation

Sample_DDPM($\varepsilon_{\mathbf{w}}$:Denoiser)

- 1: Sample *latent* $x_T \sim \mathcal{N}^0$
- 2: **for** $t = T, \dots, 1$ **do**
- 3: Pass x_t and t forward to find $\hat{\varepsilon}_t = \varepsilon_{\mathbf{w}}(x_t, t)$
- 4: Denoise the sample x_t as

$$\hat{x}_{t-1} \leftarrow \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\varepsilon}_t \right)$$

- 5: Sample $\tilde{\varepsilon} \sim \mathcal{N}^0$ and move reversely as $x_{t-1} = \hat{x}_{t-1} + \sigma_t \tilde{\varepsilon}$
- 6: **end for**
- 7: **return** Sample x_0

Few Implementational Notes on DDPM

There are a few notes: *in practice with DDPMs*

- We typically set $\bar{\omega}_t = 1$ for all t
 - ↳ This factor does **not** really make a **significant impact**
- In basic proposal, it is suggested to set $\sigma_t = \sqrt{1 - \alpha_t}$
 - ↳ This means that the variance **in both directions are the same**
 - ↳ At time $t = 1$, we set $\sigma_t = 0$, i.e., **no noise** added to **last denoising**
- The coefficients α_t in general should be **scheduled**
 - ↳ **Linear** scheduling is one classical option where $\alpha_t = \xi t + \kappa$
 - ↳ This might impact **considerably** on performance

Sampling Time of DDPM

The training of DDPM seems solid; however,

the *sampling* can take *long*

This is because the reverse diffusion is *stochastic*

↳ It needs *time to converge*

There are two simple remedies

↳ Learn more parameters for reverse trajectory, e.g., σ_t

↳ Simple extension of DDPM with *learnable variances*

↳ Make a *deterministic* sampling mechanism *for generation*

DDIM: Sampling with Zero Variance

We can use DDPM to estimate noise sample

$$\hat{\epsilon}_t = \epsilon_{\mathbf{w}}(x_t, t)$$

If this was a very *accurate estimate*, we could have said

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t \rightsquigarrow \hat{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1 - \bar{\alpha}_t}\hat{\epsilon}_t)$$

This is though *not* accurate!

Denoising Diffusion with *Implicit* Modeling $\equiv$ DDIM

We may use this estimate to *directly estimate* previous time sample x_{t-1}

DDIM: Sampling with Zero Variance

At time t , we get

$$\hat{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_{\mathbf{w}}(x_t, t) \right)$$

We could then estimate x_{t-1} as

$$\begin{aligned} \hat{x}_{t-1} &= \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \hat{\varepsilon}_{t-1} \\ &= \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \varepsilon_{\mathbf{w}}(x_{t-1}, t-1) \end{aligned}$$

We cannot compute $\varepsilon_{\mathbf{w}}(x_{t-1}, t-1)$ since we *don't* have x_{t-1} , but

- This describes approximate *ordinary* differential equation for *continuous* t
- With no randomness we can say $\varepsilon_{\mathbf{w}}(x_{t-1}, t-1) \approx \varepsilon_{\mathbf{w}}(x_t, t)$

DDIM: Sampling with Zero Variance

At time t , we get

$$\hat{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_{\mathbf{w}}(x_t, t))$$

We could then estimate x_{t-1} as

$$\hat{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \varepsilon_{\mathbf{w}}(x_t, t)$$

! Attention

DDIM **only** modifies **generation**! Training is carried out **identical** to DDPM!

DDIM Sampling

Sample_DDIM($\varepsilon_{\mathbf{w}}$:Denoiser)

- 1: Sample *latent* $x_T \sim \mathcal{N}^0$
- 2: **for** $t = T, \dots, 1$ **do**
- 3: Pass x_t and t forward to find $\hat{\varepsilon}_t = \varepsilon_{\mathbf{w}}(x_t, t)$
- 4: Find an estimate of data sample as

$$\hat{x}_0 \leftarrow \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\varepsilon}_t)$$

- 5: Denoise as $x_{t-1} \leftarrow \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \hat{\varepsilon}_t$
- 6: **end for**
- 7: **return** *Sample* x_0